SIMATS ENGINEERING

SIMATS ENGINEERING
Department of Computer Science and Engineering

CSA15 Cloud Computing and Big Data Analytics

CO3 AT2 - Capstone Cloud Access and Service Integration Case Study Student Template

Course Code	CSA15	**Course Title**	Cloud Computing and Big Data Analytics
Assessment	CO3 AT2 - Capstone Cloud Access and Service Integration Case Study	**Submission Type**	Individual digital case-study document
Faculty	Dr C. Rajesh Babu	**Employee ID**	SSETSCS415
Mode	Case study / capstone design reasoning	**Submission**	GitHub repository link through Google Classroom

How This Template Works
Read each case, connect it to the technical ideas learned in CO3 AT1, and then extend the design to your own capstone product. The answer must show cloud access, APIs, storage, serverless/event flow and edge/fog/cloud reasoning. Do not upload passwords, private keys, API tokens or sensitive personal data.

Student and Capstone Details

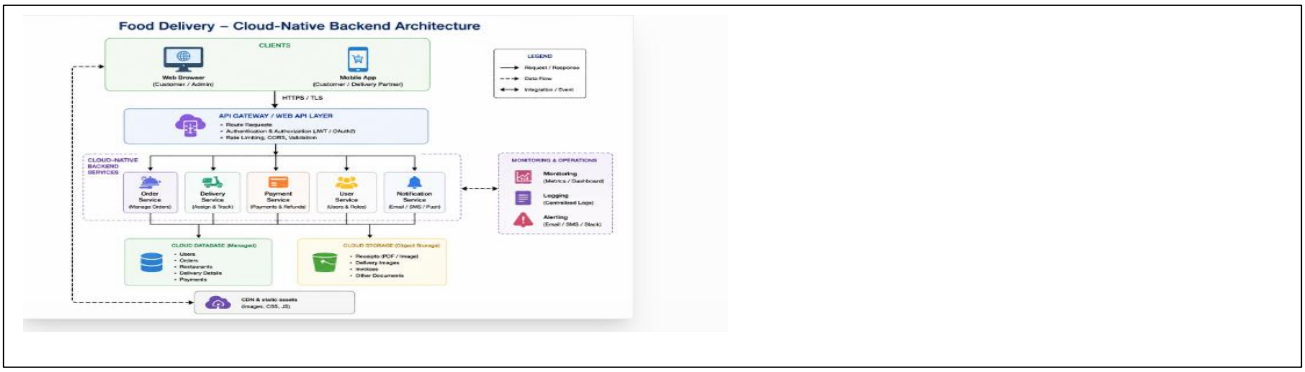
Student Name	Blessy S
Register Number	192421053
Class / Slot	Slot B
Capstone Title	Cloud security
CO3 AT1 Evidence Link	https://github.com/192421053simats-cmd/Cloud-computing-
GitHub Repository Link	https://github.com/192421053simats-cmd/Cloud-computing-
Submission Date	13.08.26

Case 1: Cloud-Native Backend and Web API Architecture

Scenario
A food delivery company wants to deploy a cloud-native backend with web APIs, browser/mobile clients and shared cloud storage for receipts and delivery images. Design the architecture and explain the role of each component.

A. Case Architecture Diagram

Type response / paste diagram / add evidence here

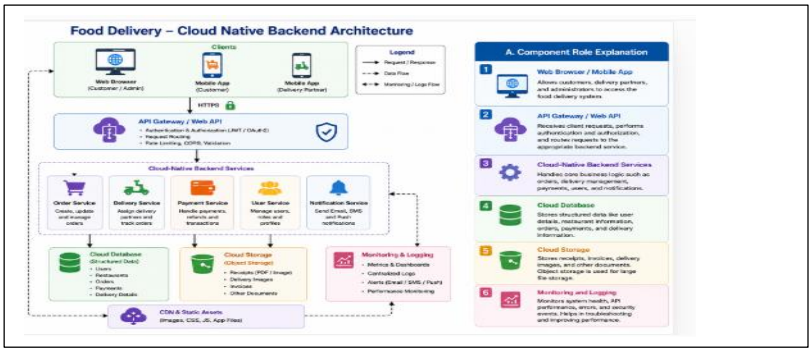


Role of Components

- **Browser/Mobile Clients:** Users place orders, track deliveries, and view receipts.
- **Web API Layer:** Receives requests and provides secure communication between clients and backend services.
- **Cloud-Native Backend:** Processes orders, payments, delivery status, and business logic.
- **Cloud Database:** Stores structured information such as users, orders, and delivery details.
- **Cloud Storage:** Stores receipts and delivery images as files.
- **Monitoring & Logging:** Tracks API requests, errors, performance, and system activity.

B. Component Role Explanation

Type response / paste diagram / add evidence here.

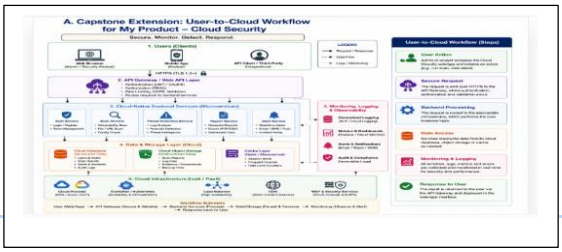


Component Role Explanation

1. **Web Browser / Mobile App:** Allows customers, delivery partners, and administrators to access the food delivery system.
2. **API Gateway / Web API:** Receives client requests, performs authentication and authorization, and routes requests to the correct backend service.
3. **Cloud-Native Backend Services:** Handles important functions such as orders, delivery tracking, payments, users, and notifications.
4. **Cloud Database:** Stores structured information such as customer details, restaurant data, orders, payments, and delivery information.
5. **Cloud Storage:** Stores receipts, invoices, delivery images, and other files. Object storage is suitable for these large files.
6. **Monitoring and Logging:** Records system activity, errors, API performance, and security events to help administrators monitor and troubleshoot the application.

C. Capstone Extension: user-to-cloud workflow for your product

Type response / paste diagram / add evidence here.



- **User Access** – The user accesses the cloud security application through a web browser or mobile device.
- **Authentication** – The system verifies the user's identity using username/password, MFA, or other authentication methods.
- **Secure Connection** – Data travels through an encrypted HTTPS/TLS connection to protect it from interception.

- **Cloud Application** – The cloud application receives user requests and processes them securely.
- **Access Control** – Role-Based Access Control (RBAC) ensures that users can access only the resources they are authorized to use.
- **Data Encryption** – Sensitive information is encrypted both while being transmitted and while stored in the cloud.
- **Cloud Storage/Database** – User data, security logs, access records, and application data are stored securely in cloud storage or databases.
- **Threat Monitoring** – Security logs, login attempts, network activity, and API requests are continuously monitored for suspicious behavior.

Case 2: Storage and Serverless Upload Workflow

Scenario
A media startup must decide between SAN-backed storage, object storage and serverless processing for upload workflows. Recommend a deployment model and justify it.

Data / Event	Best Storage	Serverless Action	Reason
User uploads a security log file	Object Storage	Automatically trigger a serverless function to scan and process the log	Object storage is scalable and suitable for large files. Serverless processing handles uploads automatically without managing servers.
User uploads an image or security evidence file	Object Storage	Trigger a function to validate, encrypt, and classify the file	Provides scalable storage and improves security through automatic processing.

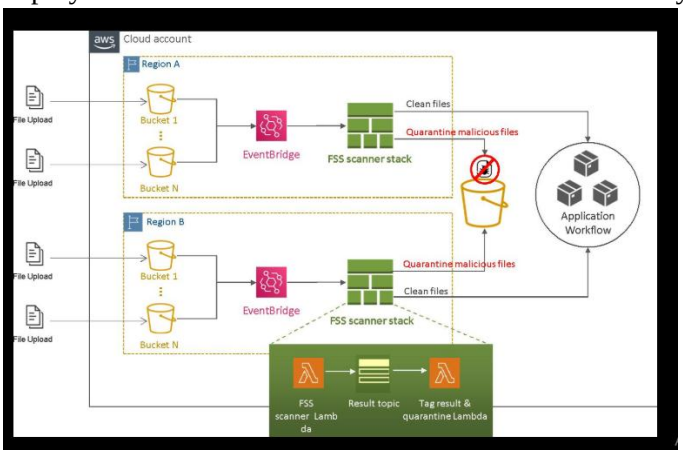
Security alert is generated	Object Storage + Database	Trigger a serverless function to record the alert and notify the administrator	Allows quick processing and storage of security events while reducing infrastructure management.
Large volumes of cloud security logs are received	Object Storage	Trigger serverless processing for filtering and threat analysis	Object storage can handle large volumes of data, while serverless functions can process events automatically.

A. Storage and Serverless Recommendation

Type response / paste diagram / add evidence here.

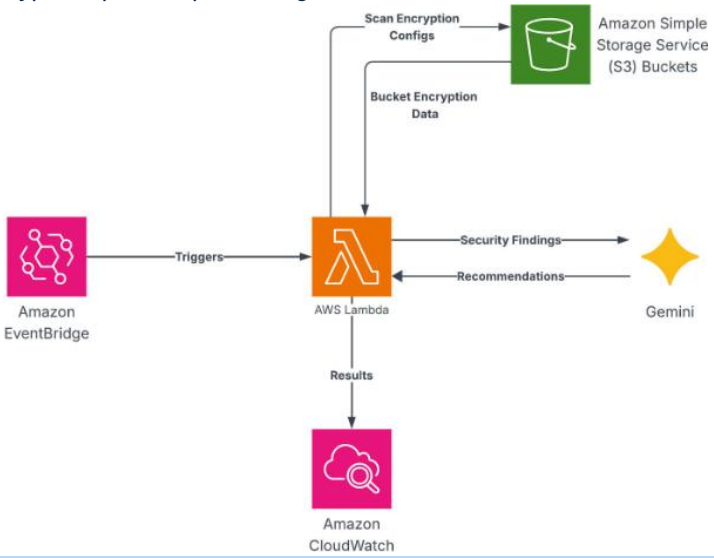
Storage and Serverless Recommendation:

For my cloud security application, I recommend using **Object Storage with Serverless Processing**. Security logs, alert records, images, reports, and other security evidence can be stored in scalable object storage such as Amazon S3 or Azure Blob Storage. When a file is uploaded or a security event occurs, a serverless function can automatically process the data without requiring a dedicated server. The function can validate, encrypt, scan, classify, and analyze the data and generate security alerts when suspicious activity is detected. This approach provides **scalability, cost efficiency, automatic processing, reduced server management, and faster threat detection**. Security controls such as encryption, access control, logging, monitoring, and authentication should be applied to protect the stored data. Therefore, combining object storage with serverless processing is a suitable deployment model for a scalable and secure cloud security application.



B. Failure Handling and Cost/Performance Explanation

Type response / paste diagram / add evidence here.

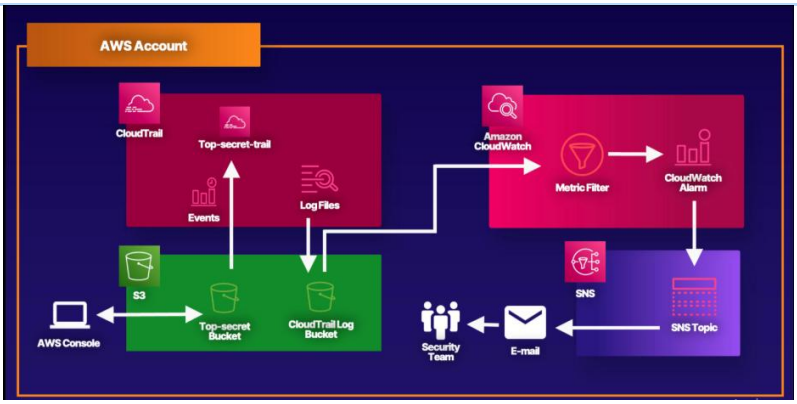


For the **Cloud Security** application, failure handling is important because security services must continue working even when a server, storage component, or processing function fails. If a serverless function fails while processing a security log or alert, the event can be retried automatically, and failed events can be sent to a dead-letter queue for later investigation. Object storage provides reliable and scalable storage, while data can be replicated or backed up to reduce the risk of data loss. Monitoring and logging should continuously track application errors, failed requests, unusual activity, and service availability. Administrators can receive alerts when repeated failures or security incidents occur.

The recommended **Object Storage + Serverless Processing** model also provides good cost and performance benefits. Object storage is suitable for large volumes of security logs, reports, images, and evidence files because storage can scale according to demand. Serverless functions execute only when required, so there is no need to continuously maintain dedicated servers. This can reduce infrastructure and maintenance costs, especially when the workload changes over time. Serverless processing can also improve performance by automatically handling multiple security events and processing them in parallel. However, serverless systems still need proper monitoring, retry policies, access control, encryption, and resource limits to prevent failures and unexpected costs.

C. Capstone Extension: one upload/event workflow

Type response / paste diagram / add evidence here.



When a user or administrator uploads a security log or evidence file, the request is first authenticated and transmitted securely using HTTPS. The file is stored in cloud object storage, where encryption and access controls protect the data. The upload event automatically triggers a serverless function. The function validates the file and analyzes its contents for suspicious patterns, unauthorized activity, or possible security threats. If a threat is detected, the system generates an alert and notifies the administrator. If no threat is found, the file is marked as safe and the analysis result is stored. The event and analysis results are also recorded in security logs and displayed on the cloud security dashboard. This workflow provides **automatic processing, scalability, secure storage, real-time threat detection, and reduced server management**.

Case 3: Edge, Fog and Cloud Placement

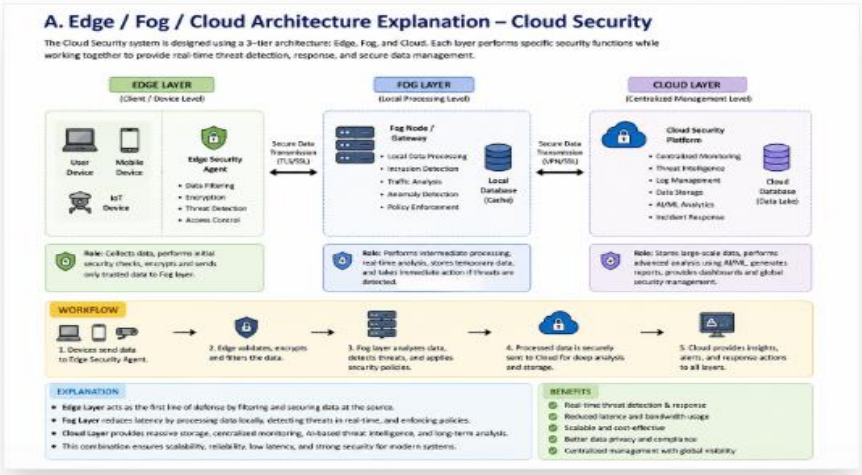
Scenario
A smart-city platform needs low-latency decision-making near sensors while still using the cloud for analytics. Decide what runs at edge, fog and cloud.

Task	Layer	Latency Need	Data Volume	Reason
Detect suspicious sensor activity	Edge	Very High	Low	Immediate detection is needed near the sensor without sending every event to the cloud.
Block unauthorized device access	Edge	Very High	Low	Security decisions must happen instantly to prevent unauthorized.

Filter and validate incoming data	Fog	High	Medium	Fog nodes can filter large amounts of data before sending important information to the cloud.
Analyze local network traffic	Fog	High	High	Fog provides faster processing close to connected devices and reduces network traffic to the cloud.

A. Edge/Fog/Cloud Architecture Explanation

Type response / paste diagram / add evidence here.



Main Cloud Security Components

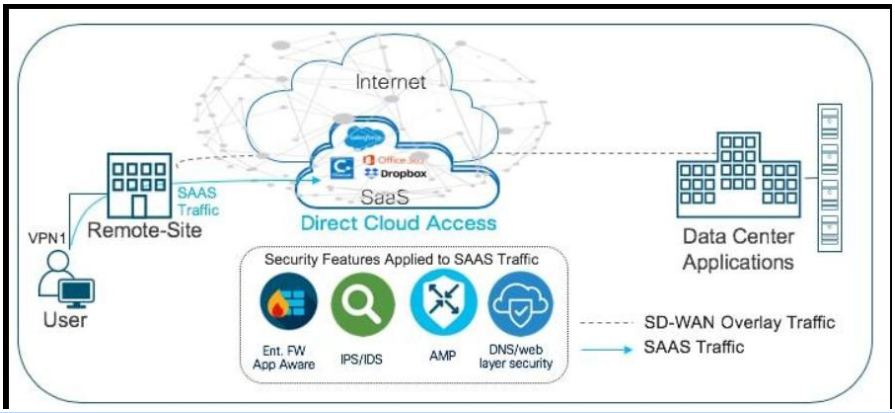
1. **Identity and Access Management (IAM)**
   Controls who can access cloud resources using authentication, authorization, and role-based access control.
2. **Data Encryption**
   Protects sensitive data by encrypting data both **at rest** and **in transit**.
3. **Network Security**
   Uses firewalls, security groups, VPNs, and network segmentation to control network traffic and prevent unauthorized connections.

4. **Threat Detection and Monitoring**
   Continuously monitors cloud activity and detects suspicious behavior, attacks, and security incidents.
5. **Backup and Disaster Recovery**
   Maintains secure backups so that data and services can be restored after failures, attacks, or accidental deletion.
6. **Security Logging and Auditing**
   Records user activities, access attempts, and system events to support monitoring, investigation, and compliance.

Cloud security is the practice of protecting cloud-based applications, data, and services from unauthorized access, cyberattacks, data loss, and other security threats. It uses **Identity and Access Management (IAM)** to control users, **encryption** to protect data, **firewalls** to secure network traffic, and **monitoring and logging** to detect suspicious activities. Backup and disaster recovery mechanisms help restore data and services during failures or attacks. These security measures ensure the **Confidentiality, Integrity, and Availability (CIA)** of cloud resources.

B. Capstone Extension or Direct Cloud Access Justification

Type response / paste diagram / add evidence here.



The system can also use **cloud backup and disaster recovery** mechanisms. Important data can be backed up regularly so that it can be restored if there is accidental deletion, hardware failure, software failure, or a cyberattack. This reduces the risk of permanent data loss and improves system reliability.

Proposed Cloud Architecture

User Device → Internet → Secure Authentication/IAM → Cloud Application → Cloud Database

Cloud Application → Monitoring & Logging

Cloud Database → Backup & Disaster Recovery

This architecture provides a centralized and secure environment for deploying the capstone project. Direct cloud access therefore extends the project from a locally accessible application into a **scalable, remotely accessible, and secure cloud-based system**. It also provides a foundation for future extensions such as mobile access, automated reporting, analytics, AI-based processing, real-time monitoring, and integration with other cloud services.

Final Capstone Cloud Access Architecture

Show Browser/Mobile Client.

Show hosted frontend or app access path.

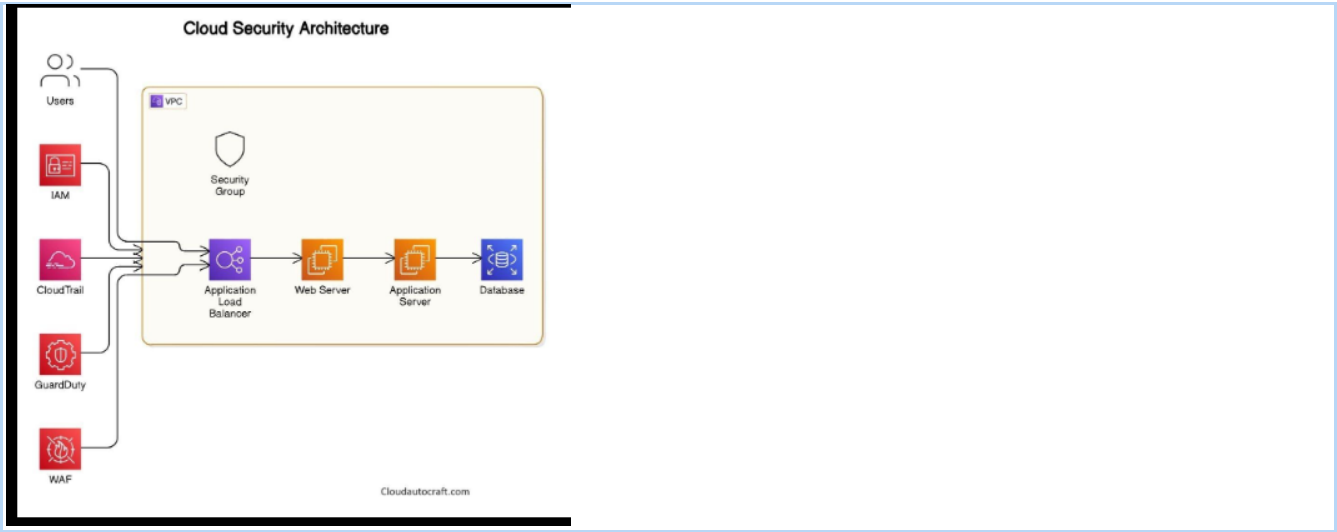
Show REST API endpoint and authentication check.
Show backend/serverless logic.

Show database, object/blob storage, logs and monitoring.

Label the arrows: request, response, upload, event, notification or monitoring signal.

Final Diagram and Explanation

Type response / paste diagram / add evidence here.



Final Cloud Architecture

Users / Devices
↓
Internet / Secure Connection
↓
Authentication & IAM
↓
Firewall & Network Security
↓
Cloud Application
↓
Encrypted Cloud Database
↓
Backup & Disaster Recovery

Cloud Application → Monitoring & Logging

Explanation

The final architecture shows a secure cloud-based system where users access the application through the internet. Authentication and **Identity and Access Management (IAM)** verify users and control their permissions. Firewall and network security mechanisms protect the cloud environment from unauthorized network access.

The **cloud application** provides the main services and processing required by the capstone project. Application data is stored in an **encrypted cloud database**, protecting sensitive information from

unauthorized access. Monitoring and logging continuously record system activities and help identify suspicious behavior or security incidents.

Regular **backup and disaster recovery** mechanisms protect important data from accidental deletion, system failures, or cyberattacks. The architecture therefore provides secure access, centralized data storage, scalability, monitoring, and reliable recovery.

Overall, the proposed architecture combines **cloud computing and cloud security** to provide a secure, scalable, and accessible environment for deploying the capstone project.

Submission Checklist

- Template completed in DOCX format.
- Diagrams pasted clearly in the document.
- GitHub repository contains DOCX/PDF, diagrams and README.
- Google Classroom submission contains GitHub link.
- Sensitive credentials are not visible anywhere.

Student Assurance

Declaration
I confirm that this case-study answer, cloud access design, diagrams and capstone extensions were prepared individually by me for the stated capstone title.

Student Signature: blessy **Date: 13.08.26**